

Fitování Weibullova a Exponenciálního Rozdělení s Cenzurovanými Daty

parc. derivace?

V tomto notebooku provádíme statistické porovnání mezi dvěma modely rozdělení — Weibullovým a Exponenciálním — pomocí datasetu s cenzurováním. Budeme:

1. Načítat a čistit data.
2. Fitovat obě rozdělení pomocí metody maximální věrohodnosti.
3. Provádět test poměru věrohodnosti pro porovnání obou modelů.
4. Vizualizovat fitovaná rozdělení.

Začněme importováním potřebných knihoven a načítáním dat.

```
# Import libraries
import pandas as pd
import numpy as np
from scipy.optimize import minimize
from scipy.stats import chi2, weibull_min, expon
from scipy.special import gamma
import matplotlib.pyplot as plt
```

Načítání a Čištění Dat

Načteme data z Excelovského souboru a vyčistíme je extrahováním relevantních sloupců: status cenzurování a pozorované časy.

```
# Load data
file_path = 'Data_2024.xlsx' # Update file path as needed
sheet_name = 'Data_věrohodnost'

# Read the relevant sheet
data = pd.read_excel(file_path, sheet_name=sheet_name)

# Clean and extract relevant columns
data_cleaned = data.iloc[:, :2]
data_cleaned.columns = ['censored', 'time']

# Convert to numpy arrays
censored = data_cleaned['censored'].values # 1 = censored, 0 = observed
time = data_cleaned['time'].values

# Display first rows of data
print("prvních 5 řádků:")
print(data_cleaned.head())
```

prvních 5 řádků:

	censored	time
0	0	6.528
1	0	6.013
2	1	6.055
3	0	7.243
4	1	5.629

Definování Funkcí Negativní Logaritmické Věrohodnosti

Definujeme funkce negativní logaritmické věrohodnosti pro Weibullovo a Exponenciální rozdělení. Tyto funkce zohledňují cenzurování v datech.

```
# Define the negative log-likelihood function for Weibull distribution
def weibull_neg_log_likelihood(params, t, delta):
    """
    Corrected Negative log-likelihood function for Weibull
    distribution with censoring.

    params: list [k, lambda], k = shape, lambda = scale
    t: array of times
    delta: array of censoring indicators (1 = censored, 0 = observed)
    """
    k, lam = params
    if k <= 0 or lam <= 0: # Ensure parameters are positive
        return np.inf

    log_f = np.log(k / lam) + (k - 1) * np.log(t / lam) - (t / lam) **
k # Log PDF
    log_S = - (t / lam) ** k # Log Survival Function

    # Combine observed and censored contributions
    log_likelihood = np.sum((1 - delta) * log_f + delta * log_S)
    return -log_likelihood # Negative for minimization
```

Fitování Weibullova Rozdělení

Nyní použijeme `scipy.optimize.minimize` k fitování Weibullova rozdělení na data pomocí optimalizace funkce negativní logaritmické věrohodnosti.

```
# Optimize Weibull parameters
initial_guess = [1.5, 6.0]
result = minimize(weibull_neg_log_likelihood, initial_guess,
args=(time, censored),
                    bounds=((1e-5, None), (1e-5, None)))

# Extract Weibull estimates
shape_est, scale_est = result.x
```

```
print(f"Odhad parametru tvaru k: {shape_est:.2f}")
print(f"Odhad parametru míry  $\lambda$ : {scale_est:.2f}")
```

Odhad parametru tvaru k: 6.17
Odhad parametru míry λ : 7.43

Fitování Exponenciálního Rozdělení

Exponenciální rozdělení je speciální případ Weibullova rozdělení, parametr tvaru $k = 1$. Nyní tento model fitujeme.

```
# Define the negative log-likelihood for exponential distribution
(k=1)
def exponential_neg_log_likelihood(lam, t, delta):
    """
    Negative log-likelihood for exponential distribution (special case
    of Weibull where k=1).

    lam: scale parameter lambda
    t: array of times
    delta: array of censoring indicators (1 = censored, 0 = observed)
    """
    if lam <= 0:
        return np.inf

    # Exponential distribution log-likelihood for censored and
    uncensored data
    log_likelihood = np.sum(delta * (-np.log(lam)) - (t / lam))
    log_likelihood += np.sum((1 - delta) * (-t / lam)) # Add
    contribution from censored data

    return -log_likelihood # Negative for minimization

# Optimize for exponential model
exp_result = minimize(lambda lam: exponential_neg_log_likelihood(lam,
time, censored),
                      [np.mean(time)], bounds=((1e-5, None),))

# Extract exponential estimate
exp_scale_est = exp_result.x[0]
print(f"Odhad exponenciálního parametru  $\lambda$ : {exp_scale_est:.2f}")

Odhad exponenciálního parametru  $\lambda$ : 41.29
```

Test Poměru Věrohodnosti

Nyní vypočítáme statistiku testu poměru věrohodnosti pro porovnání fitování Weibullova a Exponenciálního rozdělení.

```

# Calculate log-likelihoods for both models
logL_weibull = -weibull_neg_log_likelihood([shape_est, scale_est],
time, censored)
logL_exp = -exponential_neg_log_likelihood(exp_scale_est, time,
censored)

# Likelihood Ratio Test
LR_stat = -2 * (logL_weibull - logL_exp)
print(f"Statistika testu poměru věrohodnosti: {LR_stat:.2f}")

# Compare to chi-square distribution
df = 1 # Degrees of freedom (1 parameter difference between the
models)
p_value = 1 - chi2.cdf(LR_stat, df)
print(f"p-hodnota: {p_value}")

# Závěr
if p_value < 0.05:
    print("Odmítám nulovou hypotézu: Weibullovo rozdělení vyhovuje
lépe než exponenciální.")
else:
    print("Nepodařilo se odmítnout nulovou hypotézu: Exponenciální
rozdělení je dostatečné.")

Statistika testu poměru věrohodnosti: 69.43
p-hodnota: 1.1102230246251565e-16
Odmítám nulovou hypotézu: Weibullovo rozdělení vyhovuje lépe než
exponenciální.

```

Vizualizace Fitovaných Rozdělení

Nakonec vizualizujeme pozorovaná data spolu s fitovanými Weibullovým a Exponenciálním rozděleními.

```

# Plot the fitted distributions
x = np.linspace(min(time), max(time), 1000)

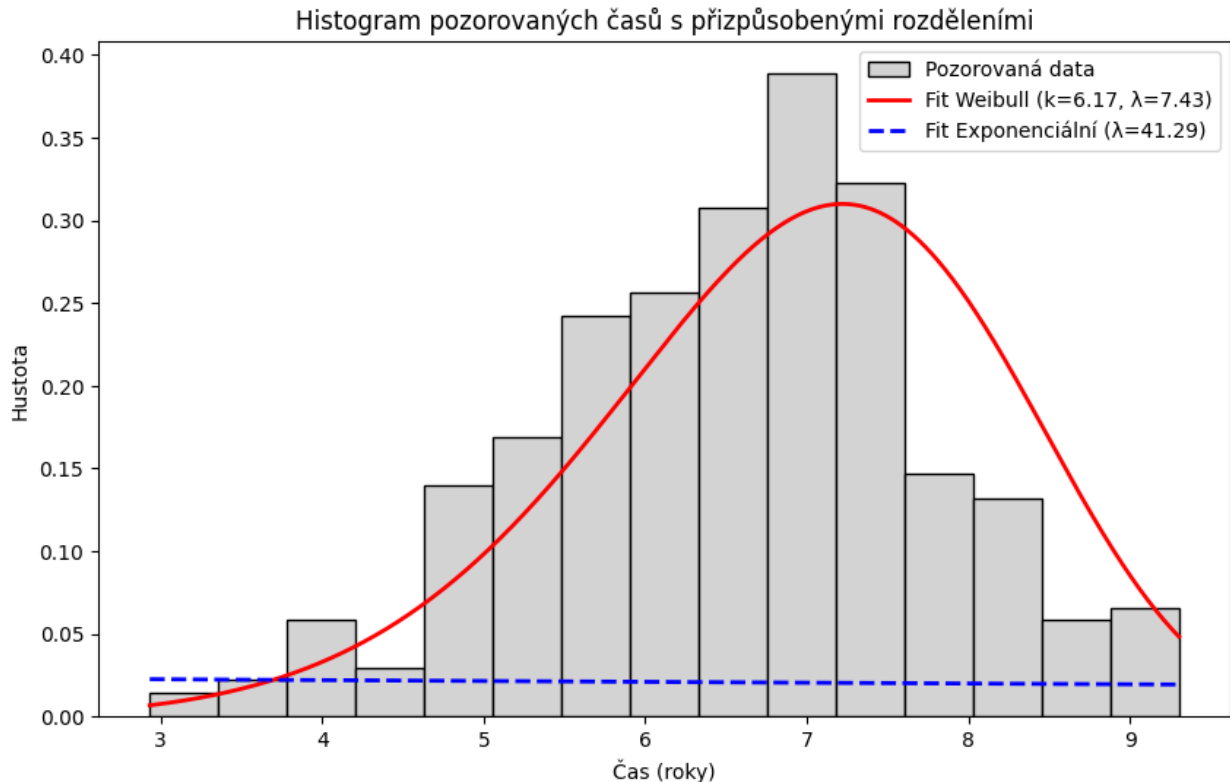
# Weibull PDF
weibull_pdf = weibull_min.pdf(x, shape_est, scale=scale_est)

# Exponential PDF
exponential_pdf = expon.pdf(x, scale=exp_scale_est)

# Vykreslení histogramu a přizpůsobených rozdělení
plt.figure(figsize=(10, 6))
plt.hist(time, bins=15, color='lightgray', edgecolor='black',
density=True, label="Pozorovaná data")
plt.plot(x, weibull_pdf, 'r-', lw=2, label=f"Fit Weibull

```

```
(k={shape_est:.2f}, λ={scale_est:.2f}))")
plt.plot(x, exponential_pdf, 'b--', lw=2, label=f"Fit Exponenciální
(λ={exp_scale_est:.2f})")
plt.title("Histogram pozorovaných časů s přizpůsobenými rozděleními")
plt.xlabel("Čas (roky)")
plt.ylabel("Hustota")
plt.legend()
plt.show()
```



Výpočet bodových odhadů pro střední dobu a percentil.

```
mean_weibull = scale_est * gamma(1 + 1/shape_est)
percentile_weibull_10 = scale_est * (-np.log(0.90))**(1/shape_est)

print(f"Střední doba pro zaměstnání v oboru: {round(mean_weibull, 2)} let")
print(f"10% percentil pro zaměstnání v oboru: {round(percentile_weibull_10, 2)} let")
```

Střední doba pro zaměstnání v oboru: 6.9 let
 10% percentil pro zaměstnání v oboru: 5.16 let

Na základě výsledků Weibullova rozdělení doby zaměstnání v oboru lze shrnout následující:

1. **Střední doba pro zaměstnání v oboru** je přibližně **6.9 let**, což znamená, že průměrná doba, po kterou absolventi zůstávají v oboru, je asi 6.9 let, než změní profesi.
2. **10% percentil** je přibližně **5.16 let**, což ukazuje, že 10 % pracovníků opustí obor do 5.16 let.

Charakteristika doby zaměstnání:

- **Tvar rozdělení ($k = 6.17$)** naznačuje, že většina pracovníků zůstává v oboru déle, s postupně rostoucí pravděpodobností odchodu.
- **Měřítka ($\lambda = 7.43$)** ukazuje, že pracovníci v oboru setrvávají v průměru déle.

Celkově Weibullovo rozdělení naznačuje, že většina pracovníků zůstává v oboru více než 5 let, ale část je schopná změnit profesi již po několika letech.

Úkol 2 *nedodržíte hierarchii + máte lin. závisl*

Krok 1: Načtení dat *prediktory: scrolling-1-interacting*

```
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt
import numpy as np
from statsmodels.stats.outliers_influence import
variance_inflation_factor

# Načtení dat
file_path = 'Data_2024.xlsx' # Změňte cestu k souboru
sheet_name = 'Data_regrese'

data = pd.read_excel(file_path, sheet_name=sheet_name)

print(data.head())
```

	OStype	ActiveUsers	InteractingPct	ScrollingPct	Ping [ms]
0	iOS	4113	0.8283	0.1717	47
1	iOS	7549	0.3461	0.6539	46
2	Windows	8855	0.2178	0.7822	55
3	Android	8870	0.0794	0.9206	56
4	MacOS	9559	0.7282	0.2718	76

Krok 2: Vytvoření modelu

```
# Add interaction term ActiveUsers_ScrollingPct
data['ActiveUsers_ScrollingPct'] = data['ActiveUsers'] *
data['ScrollingPct']
```

```

data['OSTypeIOS'] = (data['OSType'] == 'iOS').astype(int)
data['OSTypeWindows'] = (data['OSType'] == 'Windows').astype(int)
data['OSTypeMacos'] = (data['OSType'] == 'MacOS').astype(int)

# Define the new model formula
formula = 'Q("Ping [ms]") ~ ActiveUsers + InteractingPct + OSTypeIOS +
OSTypeWindows + OSTypeMacos + ActiveUsers_ScrollingPct'

# Fit the model
model = smf.ols(formula=formula, data=data)
results=model.fit()
print(results.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          Q("Ping [ms]")    R-squared:
0.785
Model:                  OLS               Adj. R-squared:
0.783
Method:                 Least Squares      F-statistic:
302.0
Date:                  Sun, 15 Dec 2024    Prob (F-statistic):
7.08e-162
Time:                  21:22:52           Log-Likelihood:
-1678.1
No. Observations:      502               AIC:
3370.
Df Residuals:          495               BIC:
3400.
Df Model:              6

Covariance Type:      nonrobust

=====
=====

```

	coef	std err	t	P> t
Intercept	11.2525	1.502	7.494	0.000
ActiveUsers	0.0024	0.000	9.666	0.000
InteractingPct	32.8854	2.457	13.382	0.000

```

-----
-----

```

OStypeIOS		-5.8093	0.913	-6.364	0.000
-7.603	-4.016				
OStypeWindows		3.5404	0.884	4.006	0.000
1.804	5.277				
OStypeMacos		9.3104	0.879	10.587	0.000
7.583	11.038				
ActiveUsers_ScrollingPct		0.0029	0.000	6.993	0.000
0.002	0.004				

```

=====
=====
Omnibus:                36.080    Durbin-Watson:
1.884
Prob(Omnibus):          0.000    Jarque-Bera (JB):
116.898
Skew:                   0.240    Prob(JB):
4.13e-26
Kurtosis:               5.315    Cond. No.
6.10e+04
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 6.1e+04. This might indicate that there are strong multicollinearity or other numerical problems.

VIF

```

# získkej VIF a slož všechny VIF do dataframe
X = pd.DataFrame(model.exog, columns=model.exog_names)
vif = pd.Series([variance_inflation_factor(X.values, i)
                 for i in range(X.shape[1])],
                 index=X.columns)
vif_df = vif.to_frame()
# Nastavení názvu sloupce
vif_df.columns = ['VIF']
print('\n\n\n')
print(vif_df)
# ukaž korelace prediktorů
print('\n\n\n')
print(X.corr())

```

	VIF
Intercept	23.801797

ActiveUsers	4.158140
InteractingPct	5.573791
OSTypeIOS	1.581669
OSTypeWindows	1.613144
OSTypeMacos	1.627265
ActiveUsers_ScrollingPct	8.399699

	Intercept	ActiveUsers	InteractingPct	
OSTypeIOS \				
Intercept	NaN	NaN	NaN	
NaN				
ActiveUsers	NaN	1.000000	0.040275	-
0.063206				
InteractingPct	NaN	0.040275	1.000000	-
0.062634				
OSTypeIOS	NaN	-0.063206	-0.062634	
1.000000				
OSTypeWindows	NaN	0.003135	-0.016964	-
0.334506				
OSTypeMacos	NaN	-0.000136	0.086466	-
0.341322				
ActiveUsers_ScrollingPct	NaN	0.582505	-0.711749	
0.011121				

	OSTypeWindows	OSTypeMacos	
ActiveUsers_ScrollingPct			
Intercept	NaN	NaN	
NaN			
ActiveUsers	0.003135	-0.000136	
0.582505			
InteractingPct	-0.016964	0.086466	-
0.711749			
OSTypeIOS	-0.334506	-0.341322	
0.011121			
OSTypeWindows	1.000000	-0.371550	
0.001067			
OSTypeMacos	-0.371550	1.000000	-
0.053109			
ActiveUsers_ScrollingPct	0.001067	-0.053109	
1.000000			

Odezva windows

```
model = model.fit()
# Select numeric predictors only for prediction
```

```

X = data[['ActiveUsers', 'InteractingPct', 'ActiveUsers_ScrollingPct',
'OStypeIOS', 'OStypeWindows', 'OStypeMacos']]

# Add a constant for the intercept term
X = sm.add_constant(X)

# Ensure all data is numeric
X = X.apply(pd.to_numeric, errors='coerce')

# Generate new predictions
X_new = pd.DataFrame({
    'ActiveUsers': np.linspace(data['ActiveUsers'].min(),
data['ActiveUsers'].max(), 100),
    'InteractingPct': data['InteractingPct'].mean(),
    'ActiveUsers_ScrollingPct': np.linspace(data['ActiveUsers'].min(),
data['ActiveUsers'].max(), 100) * data['ScrollingPct'].mean(),
    'OStypeIOS': np.ones(100), # Assuming we want to predict for
OStypeIOS = 1 (iOS)
    'OStypeWindows': np.zeros(100),
    'OStypeMacos': np.zeros(100),
})

# Add a constant for the intercept term
X_new = sm.add_constant(X_new)

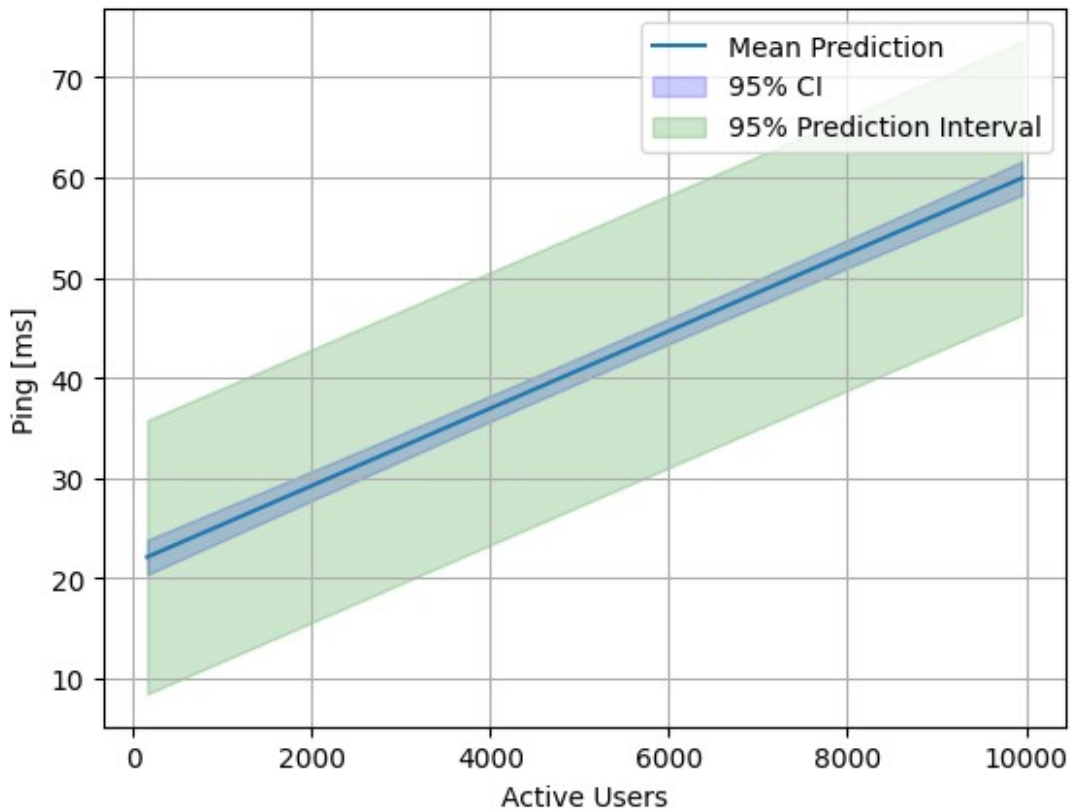
# Ensure all data is numeric
X_new = X_new.apply(pd.to_numeric, errors='coerce')

# Predict using the fitted model
predictions = model.get_prediction(X_new)
pred_summary = predictions.summary_frame(alpha=0.05) # 95% confidence
interval

# Plot the mean predictions and confidence intervals
plt.plot(X_new['ActiveUsers'], pred_summary['mean'], label='Mean
Prediction')
plt.fill_between(X_new['ActiveUsers'], pred_summary['mean_ci_lower'],
pred_summary['mean_ci_upper'], color='blue', alpha=0.2, label='95%
CI')
plt.fill_between(X_new['ActiveUsers'], pred_summary['obs_ci_lower'],
pred_summary['obs_ci_upper'], color='green', alpha=0.2, label='95%
Prediction Interval')

plt.xlabel('Active Users')
plt.ylabel('Ping [ms]')
plt.legend()
plt.grid(True)
plt.show()

```



2) Pomocí Vašeho výsledného modelu identifikujte, pro které nastavení parametrů má odezva nejproblematictější (největší) hodnotu (použijte model, nikoli samotná pozorování).

```
# Vytvoření nových dat pro všechny kombinace operačních systémů
X_new = pd.DataFrame({
    'ActiveUsers': np.linspace(data['ActiveUsers'].min(),
data['ActiveUsers'].max(), 100), # 100 různých hodnot pro ActiveUsers
    'InteractingPct': np.linspace(data['InteractingPct'].min(),
data['InteractingPct'].max(), 100), # 100 různých hodnot pro
InteractingPct
    'ActiveUsers_ScrollingPct': np.linspace(data['ActiveUsers'].min(),
data['ActiveUsers'].max(), 100) * data['ScrollingPct'].mean(),
})

# Přidání kombinací operačních systémů (pro iOS, Windows, MacOS)
X_new['OSTypeIOS'] = [1] * 100 # iOS
X_new['OSTypeWindows'] = [0] * 100 # Ne Windows
X_new['OSTypeMacos'] = [0] * 100 # Ne MacOS

# Přidání konstanty pro intercept
X_new = sm.add_constant(X_new)

# Ujistíme se, že všechny hodnoty jsou číselné
```

```

X_new = X_new.apply(pd.to_numeric, errors='coerce')

# Predikce pro iOS
predictions_ios = model.get_prediction(X_new)
pred_summary_ios = predictions_ios.summary_frame(alpha=0.05) # 95% intervaly spolehlivosti

# Najít maximální hodnotu predikce pro iOS
max_ping_value_ios = pred_summary_ios['mean'].max()
max_ping_index_ios = pred_summary_ios['mean'].idxmax()
max_ping_params_ios = X_new.iloc[max_ping_index_ios]

# Pro Windows
X_new['OSTypeIOS'] = [0] * 100 # Ne iOS
X_new['OSTypeWindows'] = [1] * 100 # Windows
X_new['OSTypeMacos'] = [0] * 100 # Ne MacOS

# Predikce pro Windows
predictions_windows = model.get_prediction(X_new)
pred_summary_windows = predictions_windows.summary_frame(alpha=0.05) # 95% intervaly spolehlivosti

# Najít maximální hodnotu predikce pro Windows
max_ping_value_windows = pred_summary_windows['mean'].max()
max_ping_index_windows = pred_summary_windows['mean'].idxmax()
max_ping_params_windows = X_new.iloc[max_ping_index_windows]

# Pro MacOS
X_new['OSTypeIOS'] = [0] * 100 # Ne iOS
X_new['OSTypeWindows'] = [0] * 100 # Ne Windows
X_new['OSTypeMacos'] = [1] * 100 # MacOS

# Predikce pro MacOS
predictions_macos = model.get_prediction(X_new)
pred_summary_macos = predictions_macos.summary_frame(alpha=0.05) # 95% intervaly spolehlivosti

# Najít maximální hodnotu predikce pro MacOS
max_ping_value_macos = pred_summary_macos['mean'].max()
max_ping_index_macos = pred_summary_macos['mean'].idxmax()
max_ping_params_macos = X_new.iloc[max_ping_index_macos]

# Výstupy
print(f"Největší hodnota predikce Ping [ms] pro iOS: {max_ping_value_ios}")
print(f"Parametry pro iOS, kde je maximální hodnota predikce:")
print(max_ping_params_ios)

print(f"Největší hodnota predikce Ping [ms] pro Windows: {max_ping_value_windows}")

```

```
print(f"Parametry pro Windows, kde je maximální hodnota predikce:")
print(max_ping_params_windows)
```

```
print(f"Největší hodnota predikce Ping [ms] pro MacOS:
{max_ping_value_macos}")
print(f"Parametry pro MacOS, kde je maximální hodnota predikce:")
print(max_ping_params_macos)
```

```
Největší hodnota predikce Ping [ms] pro iOS: 76.69826668188963
Parametry pro iOS, kde je maximální hodnota predikce:
ActiveUsers          9953.000000
InteractingPct        0.998600
ActiveUsers_ScrollingPct  5089.833344
OSTypeIOS             1.000000
OSTypeWindows         0.000000
OSTypeMacos           0.000000
Name: 99, dtype: float64
Největší hodnota predikce Ping [ms] pro Windows: 86.04795452298241
Parametry pro Windows, kde je maximální hodnota predikce:
ActiveUsers          9953.000000
InteractingPct        0.998600
ActiveUsers_ScrollingPct  5089.833344
OSTypeIOS             0.000000
OSTypeWindows         1.000000
OSTypeMacos           0.000000
Name: 99, dtype: float64
Největší hodnota predikce Ping [ms] pro MacOS: 91.81798589186373
Parametry pro MacOS, kde je maximální hodnota predikce:
ActiveUsers          9953.000000
InteractingPct        0.998600
ActiveUsers_ScrollingPct  5089.833344
OSTypeIOS             0.000000
OSTypeWindows         0.000000
OSTypeMacos           1.000000
Name: 99, dtype: float64
```

Největší hodnotu má MacOS

3) Odhadněte hodnotu odezvy uživatele s Windows, při průměrném nastavení ostatních parametrů a vypočtěte konfidenční interval a predikční interval pro toto nastavení.

```
import pandas as pd
import statsmodels.api as sm
import numpy as np

# Získání průměrných hodnot pro ActiveUsers, InteractingPct a ScrollingPct
avg_active_users = data['ActiveUsers'].mean()
avg_interacting_pct = data['InteractingPct'].mean()
avg_scrolling_pct = data['ScrollingPct'].mean()
```

```

# Vytvoření nových dat pro Windows (OSTypeWindows = 1, ostatní OS = 0)
X_new_windows = pd.DataFrame({
    'ActiveUsers': [avg_active_users],
    'InteractingPct': [avg_interacting_pct],
    'ActiveUsers_ScrollingPct': [avg_active_users *
avg_scrolling_pct], # Interakce mezi ActiveUsers a ScrollingPct
    'OSTypeIOS': [0],
    'OSTypeWindows': [1],
    'OSTypeMacos': [0]
})

# Přidání konstanty pro intercept
X_new_windows = sm.add_constant(X_new_windows)

# Ujistíme se, že všechny hodnoty jsou číselné
X_new_windows = X_new_windows.apply(pd.to_numeric, errors='coerce')

# Predikce pro Windows (konfidenční interval a predikční interval)
predictions_windows = model.get_prediction(X_new_windows)
pred_summary_windows = predictions_windows.summary_frame(alpha=0.05)
# 95% intervaly spolehlivosti

# Získání odhadu, konfidenčního intervalu (CI) a predikčního intervalu (PI)
mean_ping_value = pred_summary_windows['mean'][0]
mean_ci_lower = pred_summary_windows['mean_ci_lower'][0]
mean_ci_upper = pred_summary_windows['mean_ci_upper'][0]
obs_ci_lower = pred_summary_windows['obs_ci_lower'][0]
obs_ci_upper = pred_summary_windows['obs_ci_upper'][0]

# Výstup výsledků
print(f"Pro uživatele s Windows a průměrnými hodnotami ostatních parametrů je odhadovaná hodnota Ping [ms]: {mean_ping_value:.2f}")
print(f"Konfidenční interval (95%): [{mean_ci_lower:.2f}, {mean_ci_upper:.2f}]")
print(f"Predikční interval (95%): [{obs_ci_lower:.2f}, {obs_ci_upper:.2f}]")

Pro uživatele s Windows a průměrnými hodnotami ostatních parametrů je odhadovaná hodnota Ping [ms]: 52.03
Konfidenční interval (95%): [50.86, 53.21]
Predikční interval (95%): [38.43, 65.63]

```

4) Na základě jakýchkoli vypočtených charakteristik argumentujte, zdali je Váš model „vhodný“ pro další použití.

Na základě výsledků regresní analýzy lze učinit několik závěrů o vhodnosti modelu pro další použití:

1. **$R^2 = 0.785$** : Model vysvětluje přibližně 78,5 % variability závislé proměnné (Ping [ms]), což je dobrý výsledek, naznačující, že model má solidní schopnost predikovat.
2. **P-hodnoty**: Všechny p-hodnoty pro koeficienty (včetně `ActiveUsers`, `InteractingPct`, `OStypeIOS`, `OStypeWindows`, `OStypeMacos`, `ActiveUsers_ScrollingPct`) jsou **menší než 0,05**, což znamená, že všechny tyto prediktory jsou statisticky významné.
3. **VIF (Variance Inflation Factor)**:
 - Všechny hodnoty VIF jsou v přijatelných mezích (například `OStypeIOS` má VIF = 1.58, což ukazuje na nízkou multikolinearitu), kromě **`ActiveUsers_ScrollingPct`**, který má VIF = 8.4, což naznačuje určitou míru multikolinearity.
4. **Durbin-Watson test**: Hodnota **1.88** je v přijatelném rozmezí, což naznačuje, že není přítomná silná autokorelace mezi rezidui.
5. **Multikolinearita**: Kromě menší multikolinearity mezi `ActiveUsers_ScrollingPct` a ostatními prediktory model nevykazuje výrazné problémy s multikolinearitou.
6. **Kondiční číslo (Condition Number)**: Velké číslo **$6.1e+04$** může naznačovat problémy s numerickou stabilitou nebo silnou multikolinearitu mezi některými prediktory.

Závěr:

Model je **vhodný pro použití** s výhradou, že je třeba se zaměřit na problém **multikolinearity** mezi některými proměnnými (zejména `ActiveUsers_ScrollingPct`). Další zlepšení by mohlo zahrnovat odstranění nebo transformaci některých prediktorů, které vykazují vysokou kolineární závislost.